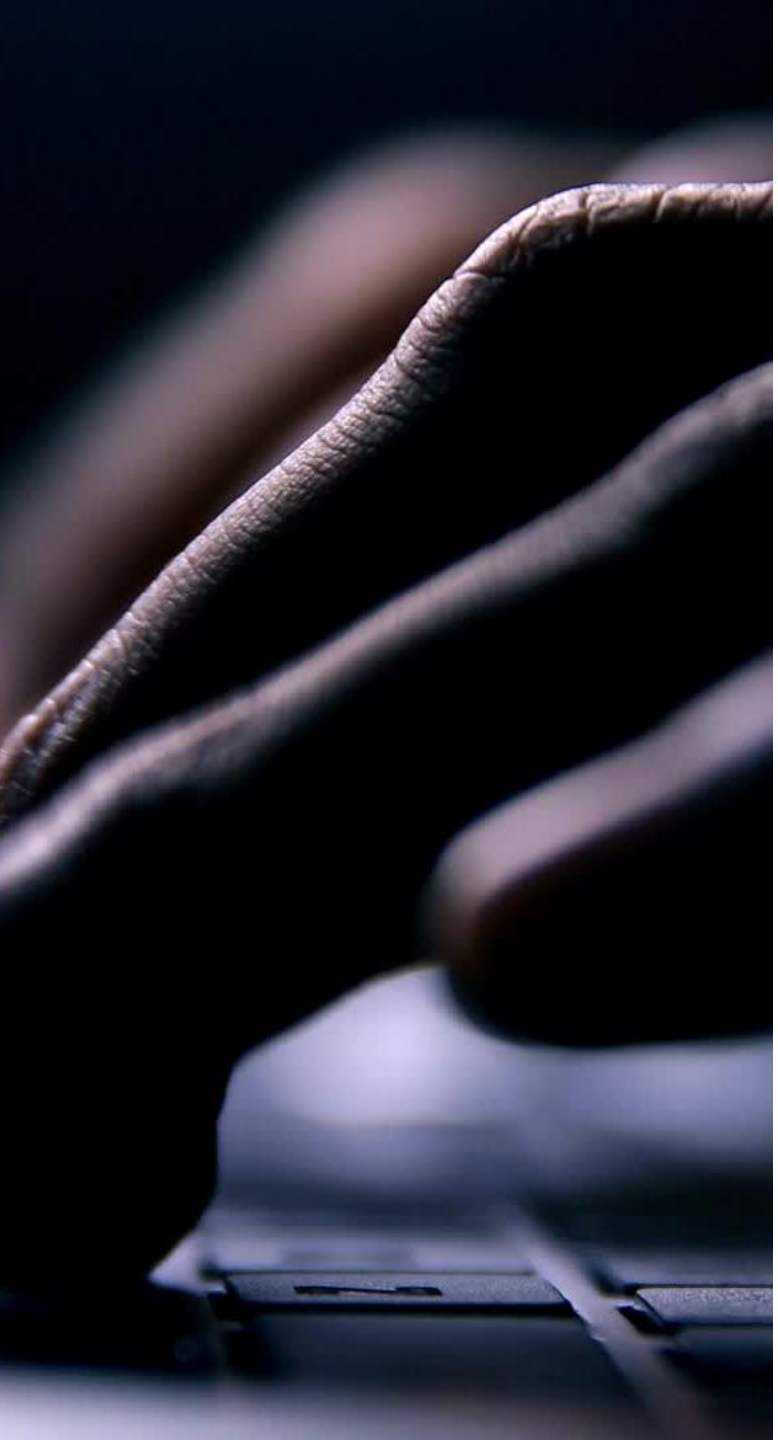


Fatec
Diadema
Luigi Papaiz

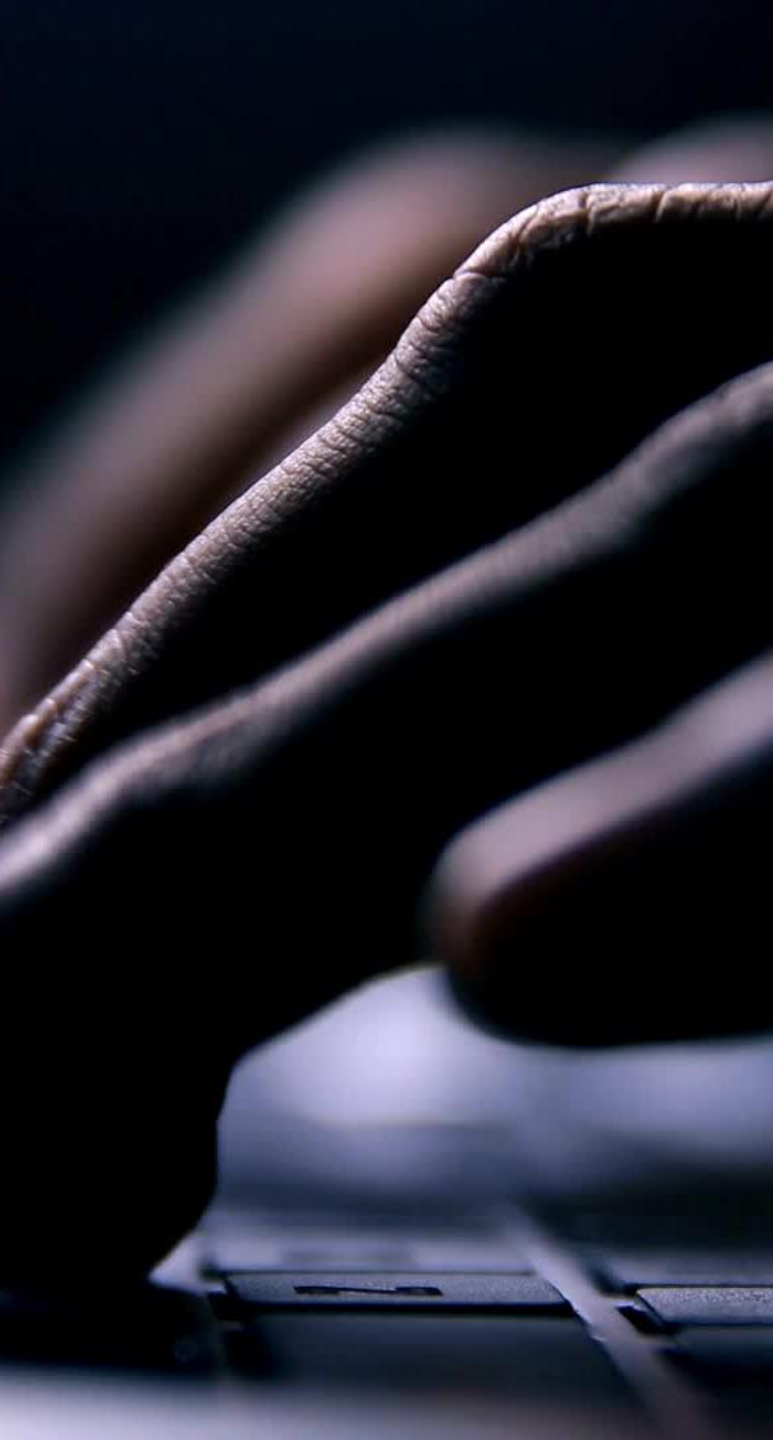
Programação para Dispositivos Móveis I

Prof. Rafael Miranda
rafael.miranda@cps.sp.gov.br

Na última aula

Fatec
Diadema
Luigi Papaiz

Classes e objetos em Kotlin



Fatec
Diadema
Luigi Papaiz

Android Studio

O que precisamos?

Fatec
Diadema
Luigi Papaiz

- Android Studio
- Android SDK
- Kotlin
- Jetpack Compose
- Emulador ou dispositivo Android

Android Studio

Fatec
Diadema
Luigi Papaiz

IDE utilizada para desenvolvimento de aplicativos Android.

Principais recursos:

- editor de código;
- compilação;
- execução;
- depuração;
- gerenciamento de dispositivos.

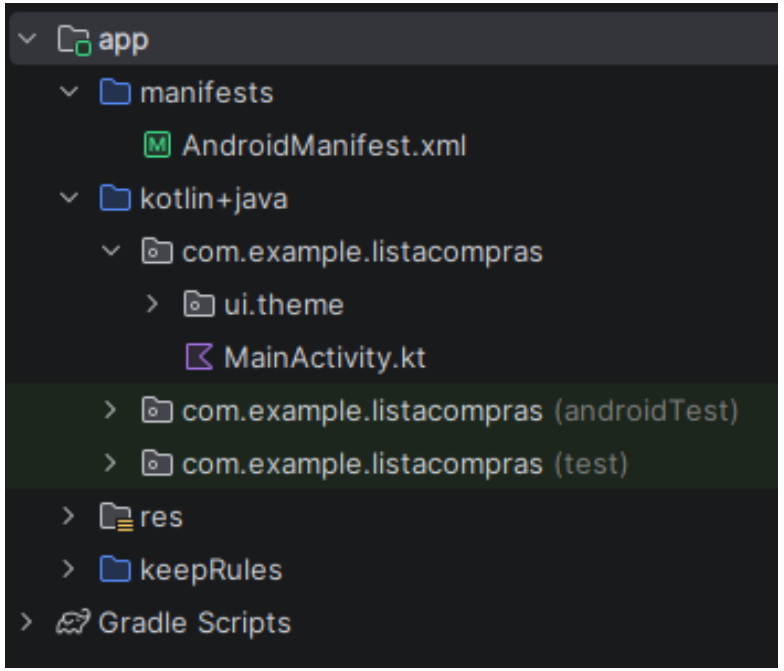
Criando nosso projeto

Fatec
Diadema
Luigi Papaiz

Android Studio > New Project > Empty Activity

Configurações:

- Name: ListaCompras
- Language: Kotlin
- demais opções: padrão



Após a criação do projeto, clicar em **RUN** para iniciar o emulador. Se não estiver disponível, será necessário subir um emulador em Device Manager.

MainActivity


```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding( paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}
```

Primeiro componente

É dentro de setContent { } que vamos construir a interface.

- Adicionar o import:

```
import androidx.compose.material3.Text
```

- E dentro do ListaComprasTheme:

```
Text("Olá, Fatec!")
```

- Código:

```
setContent {
    ListaComprasTheme {

        Text("Olá, Fatec!")

    }
}
```

Organizando componentes

- Adicionar o import:

```
import androidx.compose.foundation.layout.Column
```

- E substituir o text anterior por:

```
Column {

    Text("Lista de Compras")

    Text("Meu primeiro aplicativo Android")

}
```

Entrada de dados

- Adicionar o import:

```
import androidx.compose.material3.TextField
```

- E substituir o text anterior por:

```
Column {

    TextField(
    value = "",
    onValueChange = {}
)

}
```

Precisamos guardar o que o usuário digitou

Fatec
Diadema
Luigi Papaiz

- Adicionar o import:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
```

- Antes da Column:

```
var nome by remember {
    mutableStateOf("")
}
```

Como fica o código


```
setContent {
    ListaComprasTheme {

        var nome by remember {
            mutableStateOf("")
        }

        Column {

            Text("Lista de Compras")

            TextField(
                value = nome,
                onValueChange = {
                    nome = it
                }
            )

        }
    }
}
```

Botão

- Adicionar o import:

```
import androidx.compose.material3.Button
```

- Depois do TextField:

```
Button(
    onClick = {
        println(nome)
    }
) {
    Text("Cadastrar")
}
```

Criando o Produto

- Fora da classe MainActivity:

```
class Produto(
    val nome: String,
    val preco: Double,
    val quantidade: Int
)
```

- Dentro do onClick:

```
val produto = Produto(
    nome = nome,
    preco = 10.0,
    quantidade = 1
)

println(produto.nome)
```

Mais dados

- Criar os estados:

```
var preco by remember {
    mutableStateOf("")
}

var quantidade by remember {
    mutableStateOf("")
}
```

- Criar os campos:

```
TextField(
    value = preco,
    onValueChange = {
        preco = it
    }
)

TextField(
    value = quantidade,
    onValueChange = {
        quantidade = it
    }
)
```

Conversão

- TextField trabalha com String, mas nosso objeto precisa de Double e Int.:

```
preco.toDouble()
quantidade.toInt()
```

- O botão passa a criar:

```
Button(
    onClick = {

        val produto = Produto(
            nome = nome,
            preco = preco.toDouble(),
            quantidade = quantidade.toInt()
        )

        println(produto.nome)
    }
) {
    Text("Cadastrar")
}
```

Vários produtos

- Adicionar o import:

```
import androidx.compose.runtime.mutableStateListOf
```

```
val produtos = remember {
    mutableStateListOf<Produto>()
}
```

E isso precisa ficar **antes da Column**, junto com os estados.

Depois, no botão:

```
produtos.add(produto)
```

Mostrar os produtos

Depois da Button, ainda dentro da Column:

```
for (produto in produtos) {

    Text(
        "${produto.nome} - ${produto.quantidade} x R$
${produto.preco}"
    )

}
```

Comportamento do objeto

Na classe Produto:

```
fun calcularTotal(): Double {
    return preco * quantidade
}
```

E na tela:

```
Text(
    "${produto.nome} - " +
    "${produto.quantidade} x R$ ${produto.preco} = " +
    "R$ ${produto.calcularTotal()}"
)
```

Total da compra

Criar o estado:

```
var total by remember {
    mutableStateOf(0.0)
}
```

No botão:

```
total += produto.calcularTotal()
```

Depois do for:

```
Text("Total da compra: R$ $total")
```

Atividade — Evoluindo a Lista de Compras

Fatec
Diadema
Luigi Papaiz

O aplicativo já permite cadastrar produtos e calcular o total da compra.

Implemente:

- Limpar os campos após o cadastro.
- Adicionar textos identificando cada campo.
- Impedir o cadastro quando algum campo estiver vazio.
- Exibir a quantidade de produtos cadastrados.
- criar um botão para limpar toda a lista.